

VEKTOR

INTELLIGENCE

The Core Architecture

Hierarchical Knowledge Ecosystem — Roles, Flows & Data Model

Version 1.0 · March 16, 2026

SIG Reality Spectra · VIT Pune, Bibwewadi Campus

“Direction for every mind.”

1. What VEKTOR Intelligence Is

VEKTOR Intelligence is not a learning app with multiple user types. It is a hierarchical knowledge ecosystem where every role has a precise position in the chain, a specific set of responsibilities, and a defined relationship with every other role. The architecture is inspired by how knowledge actually flows in the real world — from researchers who discover it, to teachers who deliver it, to students who learn it — with a platform layer that governs the entire system.

The founding principle

Every teacher inherits their Domain Knowledge Graph from a researcher. Every student passes through a teacher. No one in the system operates in isolation. Knowledge has a source, a delivery mechanism, and a recipient — and VEKTOR enforces that chain at the data model level.

2. The Four Roles

VEKTOR has exactly four roles. Each role has a distinct color identity, a position in the hierarchy, and capabilities that no other role shares.

Role	Position in hierarchy	Core responsibility
Admin	Top — platform control	Security, backend, feature flags, audit logs, all users, platform statistics. The admin sees and controls everything but does not participate in learning.
Researcher	Second — knowledge source	Discovers, validates, and publishes Domain Knowledge Graphs (DKGs). Every DKG in the system originates from a researcher. Teachers inherit DKGs from researchers — they cannot create their own.
Teacher	Third — guardian layer	Receives DKGs from researchers and delivers them to students through classes. Monitors all students in their classes via real-time heatmap, individual knowledge graphs, and session history. Assigns practice tasks.
Student	Fourth — leaf node	Learns by asking questions. Each session generates a Student Knowledge Graph (SKG) which is compared against the DKG. Gaps, misconceptions, and aligned concepts are identified and fed back to the teacher automatically.

3. The Inheritance Chain

The inheritance chain is the core architectural concept. It defines how knowledge and authority flow through the system, top to bottom, and how data flows back up.

3.1 Researcher → Teacher (DKG inheritance)

A researcher publishes a Domain Knowledge Graph for a subject. This DKG contains every concept node, every edge connecting concepts, and the prerequisite relationships between them. When a teacher creates a class, they select a subject and are automatically assigned the researcher's DKG for that subject. The teacher does not modify the DKG — they only use it as the reference graph against which student understanding is measured.

Why this matters

Teachers cannot teach concepts that have not been mapped by a researcher. This ensures the knowledge graph is always academically validated before it reaches students. The researcher is the source of truth.

3.2 Teacher → Student (guardian relationship)

Every student either belongs to a teacher's class or operates in solo mode. When enrolled, the teacher becomes the student's guardian in the system. All of the student's session data — queries, tier classifications, gaps, misconceptions — is visible to the teacher in real time via Firestore onSnapshot listeners. The teacher can assign practice tasks, review individual knowledge graphs, and intervene when a student has a T3 misconception blocking their progress.

Solo mode

Students can use VEKTOR without a teacher. In solo mode, the student receives AI-generated adaptive path recommendations directly. When they later enroll in a class, all their historical session data is linked to the class retroactively.

3.3 Student → Teacher (SKG feedback)

Data does not only flow down the hierarchy. Every session a student completes generates a Student Knowledge Graph that flows back up to the teacher automatically. The teacher's heatmap aggregates misconception patterns across all students in the class. When a researcher publishes a DKG update, the change propagates down through the teacher's class to every active student session in real time.

4. The Core Session Flow

The session is the atomic unit of learning in VEKTOR. Everything in the system feeds into or out of a session. Here is exactly what happens when a student submits a query.

Item	Description
------	-------------

Step 1	Student types a STEM question into the QueryInput. The subject is auto-detected from the text. The query is sent to the FastAPI backend.
Step 2	FastAPI extracts concept entities from the query using sentence-transformers. Each concept is matched to a node ID in the Domain Knowledge Graph via semantic similarity.
Step 3	The Gemini API analyzes the query and extracted concepts to build the Student Knowledge Graph (SKG) — a graph of what the student appears to understand, believe, and not know.
Step 4	The SKG is compared against the DKG for the subject. Each concept node is classified into a tier: T1 (aligned), T2 (gap), T3 (misconception), T4 (unknown).
Step 5	The result is returned to the frontend: tier classification, gap list, misconception list, AI explanation of what went wrong, and adaptive path recommendations for what to study next.
Step 6	The session is saved to Firestore. If the student is enrolled in a class, the Firestore onSnapshot listener on the teacher's dashboard fires immediately, updating the class heatmap and student progress rings.

Live DKG updates during a session

If a researcher publishes a new DKG version while a student is mid-session, the session updates immediately via the Firestore onSnapshot listener on the DKG document. The student's gap analysis is re-run against the new version. This ensures students are always learning against the most current validated knowledge graph.

5. The Knowledge Tier System (T1–T4)

The tier system is VEKTOR's core diagnostic mechanism. It does not tell students whether they are right or wrong — it tells them what kind of gap they have. This is the fundamental difference between VEKTOR and any other learning platform.

Item	Description
T1 — Aligned	Your understanding matches the Domain Knowledge Graph. The concept node is confirmed. No intervention needed — this becomes a building block for downstream concepts.
T2 — Gap	You understand the concept at a surface level but are missing key connections to adjacent nodes in the DKG. You can answer basic questions but not advanced ones that depend on this concept.
T3 — Misconception	Your mental model has a structural error. You have a belief that is not just incomplete but actively incorrect, and it is blocking you

	from correctly understanding downstream concepts that depend on this node.
T4 — Unknown	This concept node has not appeared in any of your sessions yet. VEKTOR has no data about your understanding of it. The adaptive engine treats it as a discovery opportunity.

T3 misconceptions are the highest-priority signal in the system. A single T3 on a foundational concept can block an entire branch of the knowledge graph. The teacher's heatmap highlights T3 clusters across the class, and the adaptive path engine prioritises them above all other recommendations.

6. Firestore Data Model

VEKTOR uses four primary Firestore collections. Every connection between roles, sessions, classes, and knowledge graphs is encoded as a foreign key relationship between documents in these collections.

6.1 users/ collection

One document per user. Document ID is the Firebase Auth UID. The document contains fields for all roles plus role-specific optional fields.

```
users/{uid}

// All roles
uid:          string
email:        string
displayName:  string
role:         'student' | 'teacher' | 'researcher' | 'admin'
subjects:     string[]
levels:       Record<string, number> // 0-4 per subject
goal:         string
onboardingComplete: boolean
createdAt:    string // ISO timestamp
plan:         'free' | 'pro'

// Student-specific
classId?:    string // null = solo mode
teacherId?:  string // null = solo mode
streak?:     number
lastActiveDate?: string

// Teacher-specific
classIds?:   string[]
```

```
researcherId?:      string    // researcher they inherit DKGs from

// Researcher-specific
publishedDKGs?:     string[]   // DKG IDs they own
institution?:       string
domains?:           string[]
```

6.2 sessions/ collection

One document per session. Links back to the user, optionally to their class, and to the DKG version used. The `dkgVersion` field ensures historical sessions remain accurate even after a researcher publishes an update.

```
sessions/{sessionId}

userId:             string    // FK → users/{uid}
classId?:           string    // FK → classes/{classId} (null if solo)
subject:            string
query:              string    // the original question text
tier:               'T1' | 'T2' | 'T3' | 'T4'
conceptsAligned:    number    // T1 count
gapsFound:          number    // T2 count
misconceptions:     number    // T3 count
duration:           number    // seconds
createdAt:          string    // ISO timestamp
dkgVersion:         string    // version of DKG used in this session
skgNodes:           NodeObject[] // full student knowledge graph
gapList:            string[]   // concept IDs with gaps
misconceptionList:  string[]   // concept IDs with misconceptions
```

6.3 dkg/ collection

One document per subject. Updated by researchers when they publish a new DKG version. The `version` field enables live updates — all active sessions listen to this document via `onSnapshot`.

```
dkg/{subject}

subject:            string    // 'mathematics' | 'physics' | etc.
version:            string    // semantic version e.g. '2.1.0'
publishedAt:        string    // ISO timestamp
researcherId:       string    // FK → users/{uid}
nodeCount:          number
nodes: [
  {
    id:              string
```

```
    label:      string    // concept name
    subject:    string
    tier:        string    // difficulty level
    prerequisites: string[] // node IDs that must be T1 first
    description: string
  }
]
edges: [
  { source: string, target: string, weight: number }
]
```

6.4 classes/ collection

One document per class. Owned by a teacher. Contains student IDs, the DKG being used, assigned tasks, and a heatmap cache that is updated every time a student session completes.

```
classes/{classId}

teacherId:      string    // FK → users/{uid}
name:           string    // class display name
subject:        string
dkgId:          string    // FK → dkg/{subject}
studentIds:     string[]  // FK[] → users/{uid}
createdAt:      string
tasks: [
  {
    id:          string
    title:        string
    subject:      string
    dueDate:      string
    type:         'practice' | 'quiz' | 'review'
    conceptIds:   string[]  // which DKG nodes to practice
    createdAt:    string
  }
]
heatmapCache: {
  [conceptId: string]: {
    t1Count: number, // students aligned on this concept
    t2Count: number, // students with gap
    t3Count: number, // students with misconception
    lastUpdated: string
  }
}
```

7. Firestore Security Rules

Security rules enforce the hierarchy at the database level. No client-side logic should be relied upon for access control — the rules are the authoritative boundary.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Users: read own doc, admin reads all
    match /users/{uid} {
      allow read: if request.auth.uid == uid || isAdmin();
      allow write: if request.auth.uid == uid || isAdmin();
    }

    // Sessions: student reads own, teacher reads class, admin reads all
    match /sessions/{sessionId} {
      allow read: if isOwnSession() || isTeacherOfClass() || isAdmin();
      allow create: if request.auth.uid == request.resource.data.userId;
      allow update: if isAdmin();
    }

    // DKGs: all authenticated users read, only researchers write
    match /dkg/{subject} {
      allow read: if request.auth != null;
      allow write: if isResearcher() || isAdmin();
    }

    // Classes: teacher reads/writes own, students read their class
    match /classes/{classId} {
      allow read: if isTeacherOfClass() || isStudentInClass() || isAdmin();
      allow write: if isTeacherOfClass() || isAdmin();
    }

    function isAdmin() {
      return
      get(/databases/{database}/documents/users/{request.auth.uid}).data.role ==
      'admin';
    }
    function isResearcher() {
      return
      get(/databases/{database}/documents/users/{request.auth.uid}).data.role ==
      'researcher';
    }
    function isOwnSession() {
      return request.auth.uid == resource.data.userId;
    }
    function isTeacherOfClass() { ... }
    function isStudentInClass() { ... }
  }
}
```

8. Real-Time Connections

VEKTOR uses Firestore onSnapshot listeners to keep dashboards live without polling. The following listeners are active at different points in the system.

Item	Description
Student dashboard	Listens to own user document for streak and plan updates. Listens to sessions/ filtered by userId for recent sessions and DKG progress rings.
Session page	Listens to dkg/{subject} for live DKG version updates. If a researcher publishes mid-session, the session re-runs the gap analysis immediately.
Teacher heatmap	Listens to classes/{classId} for heatmapCache updates. Every time a student session completes and writes to Firestore, the heatmap updates within 2 seconds.
Teacher student graph	Listens to sessions/ filtered by classId for real-time student session activity. Teacher sees new sessions appear in their class history live.
Researcher DKG dashboard	Listens to sessions/ filtered by dkgVersion to see how students are performing against each version of their published graph.
Admin platform stats	Listens to aggregate counters in a stats/ collection that are updated by Cloud Functions on session write.

9. The AI Stack

VEKTOR's intelligence layer sits in the FastAPI backend on Railway. It does not use a single AI — it uses a layered stack where each component does what it is best at.

Item	Description
sentence-transformers	Local model. Runs on the Railway server. Converts concept text into dense vector embeddings. Used to match concepts extracted from a student query to node IDs in the DKG via cosine similarity. Fast, offline, no API cost per request.
NetworkX	Python graph library. Holds the DKG in memory as a directed graph. Used to find prerequisite chains, identify which T3 misconceptions are blocking downstream nodes, and compute the adaptive path recommendations.
Gemini API (primary)	Google's multimodal model. Used for the heavy lifting: analyzing the student query, building the SKG, generating the AI explanation of the misconception, and generating practice questions. Called once per session.

Claude API (fallback)	Anthropic's model. Called if Gemini API is unavailable or returns an error. Same prompt, same expected output format. The fallback is transparent to the student.
FastAPI	The orchestration layer. Receives the session request, calls sentence-transformers for concept extraction, calls Gemini for SKG generation, runs NetworkX for gap analysis, and returns the complete session result to the frontend.

10. Key Design Decisions

10.1 Why hierarchical inheritance instead of flat roles

Most platforms treat roles as independent permission sets. VEKTOR treats roles as positions in a knowledge delivery chain. A teacher's effectiveness is literally measured by their students' T3 resolution rate. A researcher's DKG quality directly determines what every teacher and student in the system can learn. The hierarchy is not organizational — it is epistemological.

10.2 Why the DKG lives in Firestore, not a JSON file

A static JSON file cannot be updated live, versioned per researcher, or listened to via onSnapshot. By storing the DKG as a Firestore document, every update by a researcher propagates to every active session within seconds. The version field on every session document means historical analysis remains accurate even as the DKG evolves.

10.3 Why solo mode is allowed

Requiring a teacher would create a chicken-and-egg problem for early adoption. Students who sign up before their school adopts VEKTOR should still get the full learning experience. Solo mode provides AI-generated adaptive path recommendations instead of teacher-assigned tasks. When the student later enrolls in a class, all their session history is preserved.

10.4 Why T3 is the highest-priority tier

A T2 gap means the student hasn't encountered something yet — this is normal and expected. A T3 misconception means the student has a wrong belief that is actively interfering with their ability to learn downstream concepts. Left unaddressed, a single T3 on a foundational node (like eigenvalue decomposition in mathematics) can block an entire branch of advanced concepts. The system treats T3 as urgent.

10.5 Why the heatmap cache lives in the class document

Computing the heatmap on-demand from raw session data would require reading hundreds of session documents on every teacher dashboard load. By maintaining a precomputed heatmapCache in the class document that is updated on every session write, the teacher's heatmap is a single document read that returns in milliseconds regardless of class size.

11. What This Architecture Enables

- **A researcher in Pune can publish a DKG update and every student in India studying that subject sees updated gap analysis within seconds.**
- A teacher can see which specific concept in the DKG is causing the most T3 misconceptions across their entire class — without reading a single session transcript.
- A student's adaptive path is not generated by a generic recommendation algorithm — it is computed from the precise structural gaps in their individual knowledge graph relative to the DKG.
- Every piece of data in the system is traceable back to a source — a session has a DKG version, a DKG version has a researcher, a class has a teacher, a teacher has a researcher.
- Adding a new subject to VEKTOR requires one action: a researcher publishes a DKG for that subject. Everything else — teacher assignment, student gap analysis, adaptive paths — works automatically.

End of Core Architecture Document

VEKTOR Intelligence · SIG Reality Spectra · VIT Pune · v1.0